

Salim Shrestha Portfolio

React Portfolio Application Documentation

Vite + React 18 + TailwindCSS

Owner: Salim Shrestha

Role: Web Developer / Computer Engineer

Location: Kathmandu, Nepal

Repository: `react-portfolio`

Document Date: May 26, 2026

Contents

1	Project Overview	2
2	Owner Details	2
3	Technology Stack	2
4	Repository Structure	2
5	Application Architecture	3
5.1	Component Responsibilities	3
6	Data Flow	4
7	User Interface Features	4
8	Build and Run Instructions	4
8.1	Install Dependencies	4
8.2	Run Development Server	5
8.3	Build Production Files	5
8.4	Preview Production Build	5
9	Environment Configuration	5
10	Blank Page Issue and Resolution	5
11	Prompt Compliance Summary	6
12	Conclusion	6

1 Project Overview

The Salim Shrestha portfolio is a clean, modern single-page web application implemented with Vite, React 18, and TailwindCSS. It presents Salim's personal profile, technical skills, GitHub projects, GitHub public statistics, and contact links in a responsive portfolio interface.

The app is designed as a client-side React application. It uses pre-seeded profile information for stable personal details and fetches live public GitHub data on first page load. If the GitHub API is unavailable, the application falls back to static data so that the portfolio remains usable.

2 Owner Details

Name	Salim Shrestha
Title	Web Developer / Computer Engineer
Location	Kathmandu, Nepal
Email	salim9shrestha@gmail.com
Photo	https://www.ssanu.com.np/images/salim.jpg
CV	https://www.ssanu.com.np/images/Salim-Shrestha-cv.pdf
GitHub	https://github.com/salimshre
LinkedIn	https://www.linkedin.com/in/salimshrestha
Facebook	https://www.facebook.com/salimshre
Instagram	https://www.instagram.com/salimsshrestha/
YouTube	https://www.youtube.com/c/technodynamic

3 Technology Stack

Frontend Library	React 18
Build Tool	Vite
Styling	TailwindCSS
Language	Plain JavaScript and JSX
Icons	React Icons
Data Fetching	Native <code>fetch</code> inside a custom React hook
Package Manager	npm

4 Repository Structure

```
react-portfolio/  
  index.html
```

```
package.json
vite.config.js
tailwind.config.js
postcss.config.js
src/
  main.jsx
  App.jsx
  index.css
  components/
    Navbar.jsx
    Hero.jsx
    About.jsx
    Skills.jsx
    Projects.jsx
    GitHubStats.jsx
    Contact.jsx
    Footer.jsx
  hooks/
    useGitHub.js
  Documentation/
    portfolio_documentation.tex
```

5 Application Architecture

The application starts at `src/main.jsx`, where React mounts `App` into the `root` element defined in `index.html`. The `App` component coordinates global page behavior, including:

- loading GitHub data through `useGitHub`;
- applying the persisted light or dark theme;
- tracking the active page section for scroll-spy navigation;
- rendering the page sections in the required single-page order.

5.1 Component Responsibilities

`Navbar.jsx`

Fixed navigation bar, section links, active section highlight, and dark mode toggle.

`Hero.jsx`

Full-screen hero with name, location, animated role text, avatar, hire button, and CV link.

`About.jsx`

Short profile paragraph, professional title, graduation year context, and location badge.

`Skills.jsx`

Skill list with animated progress bars.

<code>Projects.jsx</code>	GitHub repository cards with language, stars, description, and external links.
<code>GitHubStats.jsx</code>	Public repository, follower, and following counters.
<code>Contact.jsx</code>	Email link and social profile icon links.
<code>Footer.jsx</code>	Footer text required by the project prompt.
<code>useGitHub.js</code>	Shared GitHub API fetch logic, loading state, error state, and fallback data.

6 Data Flow

The custom hook `useGitHub` fetches data from the public GitHub API:

- <https://api.github.com/users/salimshre>
- https://api.github.com/users/salimshre/repos?sort=updated&per_page=6

The user endpoint provides avatar, biography, repository count, followers, and following count. The repository endpoint provides the six most recently updated repositories, including name, description, language, star count, and GitHub URL.

The hook initializes the UI with fallback values and then replaces them with live data when the API request succeeds. If the request fails, the app displays the fallback profile and project data and shows an explanatory message in the Projects section.

7 User Interface Features

- Single-page layout with smooth scrolling.
- Fixed navigation with active section highlighting.
- Full-screen hero section with animated role typewriter text.
- Responsive design for mobile, tablet, and desktop screens.
- Dark mode toggle persisted in `localStorage`.
- Skeleton loading cards while GitHub data is being fetched.
- Graceful GitHub API fallback behavior.
- Contact section with email and social links.

8 Build and Run Instructions

8.1 Install Dependencies

```
npm install
```

8.2 Run Development Server

```
npm run dev
```

The local Vite development server is normally available at:

```
http://127.0.0.1:5173
```

8.3 Build Production Files

```
npm run build
```

The production output is generated in the `dist/` directory.

8.4 Preview Production Build

```
npm run preview
```

9 Environment Configuration

The application can optionally use a GitHub token to raise GitHub API rate limits. Create a local `.env` file and define:

```
VITE_GITHUB_TOKEN=your_github_token_here
```

This file should remain local and must not be committed to version control.

10 Blank Page Issue and Resolution

During development, the app displayed a blank white page while the Vite server was running. The cause was a Vite configuration setting that disabled dependency optimization in development:

```
optimizeDeps: command === "serve" ? { disabled: "dev" } : {}
```

With dependency optimization disabled, the browser received raw CommonJS React modules such as `module.exports`, which are not valid in the browser's native ES module runtime. This prevented React from mounting and resulted in a blank page.

The fix was to remove the custom `optimizeDeps` override from `vite.config.js`. Vite now pre-bundles React dependencies correctly and serves browser-compatible modules from `/node_modules/.vite/deps/`.

11 Prompt Compliance Summary

Requirement	Status	Notes
Vite + React 18	Complete	Implemented with functional components.
TailwindCSS styling	Complete	<code>src/index.css</code> contains Tailwind directives only.
GitHub public API data	Complete	User and repository endpoints are fetched.
Fallback on GitHub errors	Complete	Static user and project data are used on failure.
Skeleton loaders	Complete	Projects and stats display loading placeholders.
Single-page sections	Complete	Hero, About, Skills, Projects, GitHub Stats, Contact, and Footer are present.
Dark mode toggle	Complete	Theme is persisted with guarded <code>localStorage</code> usage.
Smooth scroll-spy navigation	Complete	Intersection Observer tracks active sections.
Responsive layout	Complete	Tailwind mobile-first classes are used throughout.
Footer text	Complete	Footer matches the required 2025 wording.
README setup notes	Complete	Includes setup, build, preview, and optional token notes.

12 Conclusion

The repository implements the requested portfolio application with the expected React component structure, GitHub API integration, fallback handling, responsive TailwindCSS layout, dark mode, and documentation. The development server blank page issue was resolved by restoring Vite's default dependency optimization behavior.